

Strings:- Sequence of characters.

OR
Whatever we put inside double quotes.

Ex-

"Tejas"

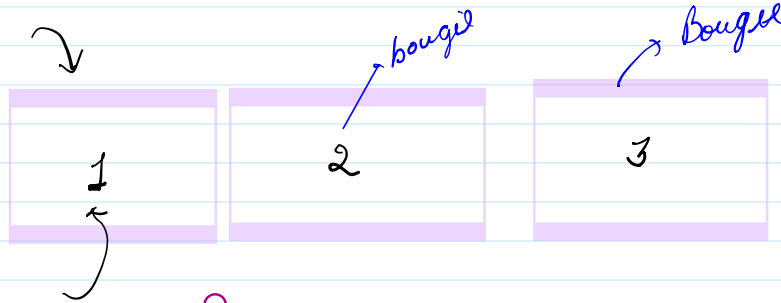
"Azaan"

"24"

"#_Azaan"

→ strings are immutable ^{that cannot change.}

Index:



S O P H I E

index → 0 1 2 3 4 5

You access a particular character using a i

Why the indexing always starts with 0 ?

index = How much in terms of distance a particular character is differ from first character.

```
name="Sophie" # p--> a
_change_name="Soahie"
```

String slicing:-

name = "Azaan Suhail"

0 1 2 3 4 5 6 7 8 9 10

first name sur name

name (start : end : step)

Inclusive starting index

exclusive

Azaan . Suhail

0 1 2 3 4 5 6 7 8 9 10

print(name[0:5])

↓

Azaan.

String formatting :-

name = "Tavishi"

print("Hello", name)

O/P → Hello Tavishi

name = "Tavishi" print(name)

```
name="Tavishi"
print("Hello",name)
```

String formatting

```
_name="Azaan"
```

```
age=24
```

```
profession="Software Engineer"
```

```
working="Remotely"
```

```
print(_name,"is a tutor at Codeyoung whose age is",age,"he is working",working,"as a", profession)
```

#String formatting using f string

```
print(f"{_name} is a tutor at Codeyoung whose age is {age} he is working {working} as a {profession}")
```

Data Types

Imagine a bank account.

A bank account contains different kinds of information:

- Account Holder Name → "Shanmuka"
- Account Balance → 50000.75
- Account Active → True
- Transaction IDs → [101, 102, 103] → Collection

All these values are different.

Python needs to know:

- Which value is text?
- Which value is a number?
- Which value is a yes/no answer?
- Which value is a collection?

This is where **Data Types** become important.

variables
↑
What kind of data it stores →

Variables:-

name = "Sophie" ✓
age = 16 ✓
wt = 45.75 ✓

Data Types

Single Value

These store one value at a time.

Examples:

- int ✓ → age → 12
- float ✓ → wt → 42.35
- str ✓ → name → "Tejas"
- bool ✓ → isactive → false

age = 23 → unordered

that can change

Collection Data Types

These store multiple values together.

Examples:

- list
- tuple
- set
- dict

you can access by Index

	Data Type	Example	Mutable/Immutable	Ordered/Unordered	Real-Life Example
single	int	100	Immutable	N/A → Not Applicable	Roll Number
	float	99.5	Immutable	N/A	Product Price
	str	"Python"	Immutable	Ordered	Student Name
	bool	True	Immutable	N/A	Login Status
collection	list	[1,2,3]	Mutable	Ordered	Shopping Cart
	tuple	(1,2,3)	Immutable	Ordered	Coordinates
	set	{1,2,3}	Mutable	Unordered	Unique IDs
	dict	{'name': 'Rahul'}	Mutable	Insertion-Ordered	Student Record